

Engenharia Informática

Arquitetura de Computadores

Informática de Gestão

Arquitetura e Funcionamento de Computadores

Docente

Carlos Manuel Gonçalves Antunes

Carlos.Antunes@islasantarem.pt

Sumário

- Registo
- Interface
- Estrutura
- Exercícios

Estrutura

MIPS (MIPS64) -Registos

- 32 registos inteiros de uso geral (GPRs-General PurposeRegisters) de 64 bits (R0..R31)
 - R0 é sempre “0” e pode ser utilizado por uma função para a qual se pretenda descartar o resultado ou como fonte de dados quando é necessário o valor 0
 - R31 é utilizado implicitamente pelas instruções JAL, BLTZAL, BLTZALL, BGEZAL e BGEZALL. Pode também ser utilizado como um registo "normal"
- 32 registos de vírgula flutuante (FPRs–FloatingPointRegisters) de 64 bits (F0..F31)
 - Se o bit “FR” no registo “CP0 Status” tem o valor “1”, 64 bits
 - Se o bit “FR” no registo “CP0 Status” tem o valor “0”, 32 bits
- PC (ProgramCounter) –endereço da instrução “seguinte”
- Registos internos

Estrutura

MIPS (MIPS64) -Interface

The screenshot displays the WinMIPS64 MIPS64 Processor Simulator interface. The main window is titled "WinMIPS64 - MIPS64 Processor Simulator - D:\mirac\terminals". The interface is divided into several panes:

- Cycles:** Shows the instruction "ld r4,A(r0)" and the pipeline stage "IF".
- Registers:** Lists registers R0 through R21 and F0 through F21, all containing zeros.
- Statistics:**
 - Execution:** 0 Cycles, 0 Instructions.
 - Stalls:** 0 RAW Stalls, 0 WAW Stalls, 0 WAR Stalls, 0 Structural Stalls, 0 Branch Taken Stalls, 0 Branch Misprediction Stalls.
 - Code size:** 40 Bytes.
- Pipeline:** A diagram showing the pipeline stages: IF (Instruction Fetch), ID (Instruction Decode), EX (Execute), MEM (Memory Access), and WB (Write Back). The EX stage is highlighted, showing the Multiplier, FP Adder, and DIV 0 units.
- Data:** Shows memory locations and their contents:
 - 0000: 0000000000000000a A: .word 10
 - 0008: 00000000000000008 B: .word 8
 - 0010: 00000000000000000 C: .word 0
 - 0018: 00000000000010000 CR: .word32 0x10000
 - 0020: 00000000000010008 DR: .word32 0x10008
- Code:** Shows the assembly code:

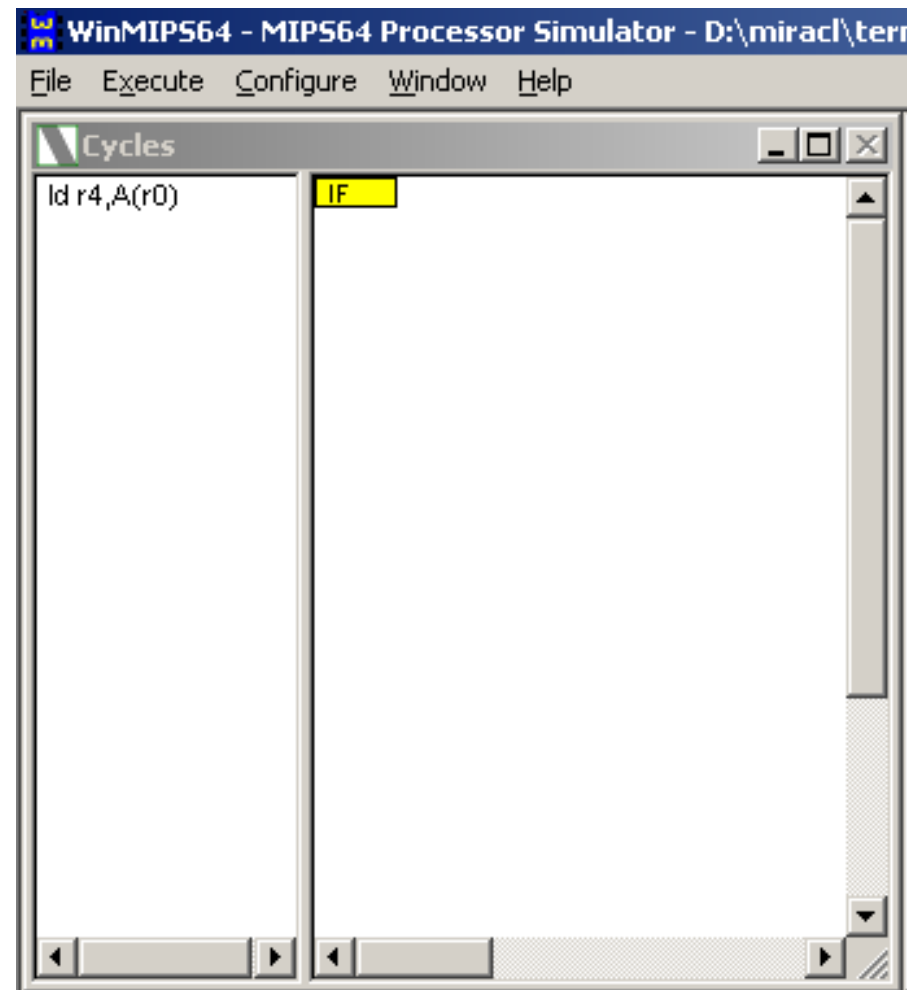

```
0000 dc040000 ld r4,A(r0)
0004 dc050008 ld r5,B(r0)
0008 0085182c dadd r3,r4,r5
000c fc030010 sd r3,C(r0)
0010 8c010018 lwu r1,CR(r0) ;Control F
0014 8c020020 lwu r2,DR(r0) ;Data Regi
0018 600a0001 daddi r10,r0,1
001c fc430000 sd r3,(r2) ;r3 output
0020 fc2a0000 sd r10,(r1) ;.. to sci
0024 04000000 halt
0028 00000000
002c 00000000
0030 00000000
0034 00000000
0038 00000000
003c 00000000
0040 00000000
0044 00000000
0048 00000000
004c 00000000
0050 00000000
```
- Terminal:** A terminal window at the bottom left, currently empty.

The status bar at the bottom indicates "Ready".

Estrutura

MIPS (MIPS64) -Interface

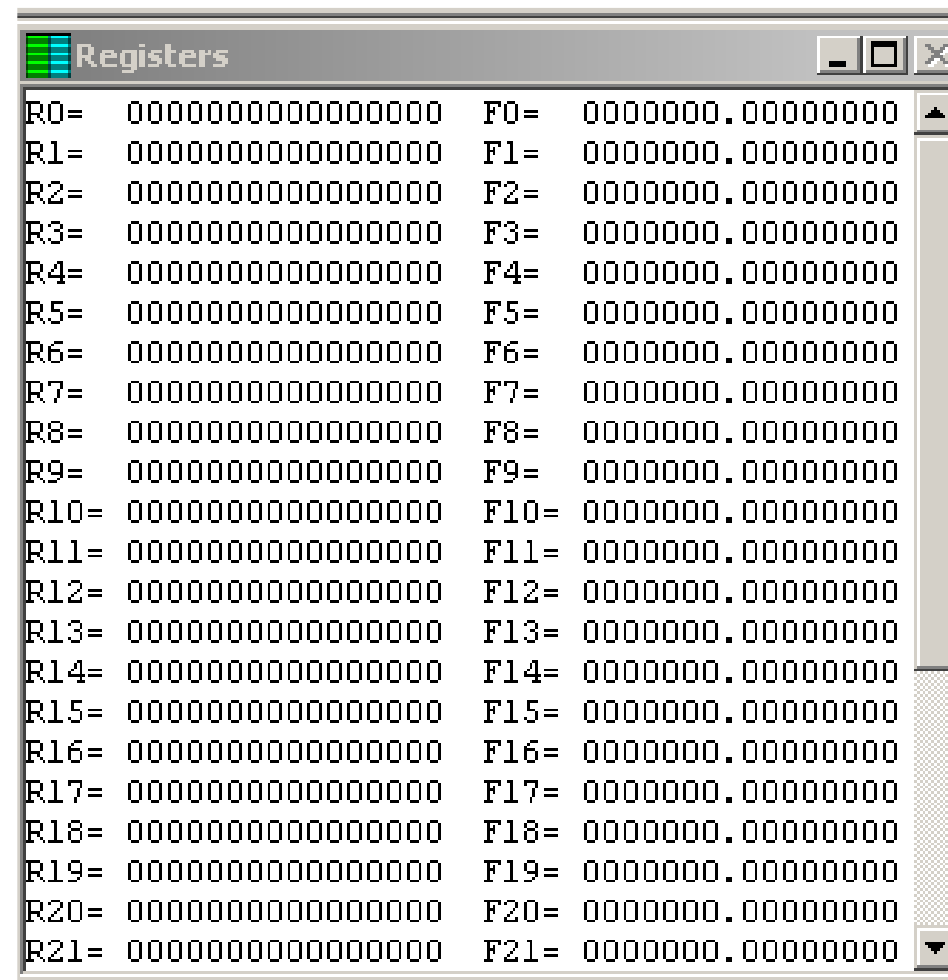
Esta janela apresenta uma disposição temporal do estado de cada instrução dentro do pipeline



Estrutura

MIPS (MIPS64) -Interface

Nesta janela são apresentados os valores dos Registos (64bits). Pode aparecer a cinzento (irá ser escrito) ou com cores (representa o estado de cada instrução); Podem ser alterados

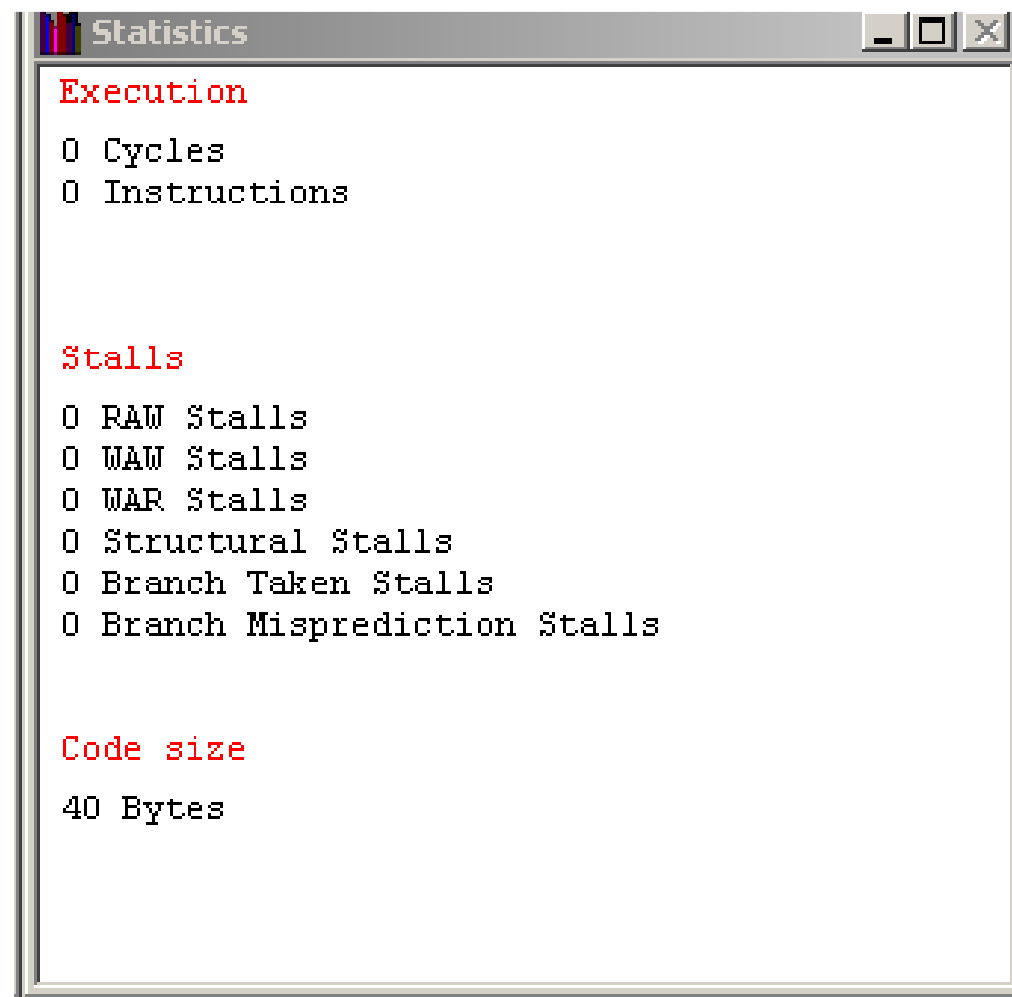


Registers			
R0=	0000000000000000	F0=	0000000.00000000
R1=	0000000000000000	F1=	0000000.00000000
R2=	0000000000000000	F2=	0000000.00000000
R3=	0000000000000000	F3=	0000000.00000000
R4=	0000000000000000	F4=	0000000.00000000
R5=	0000000000000000	F5=	0000000.00000000
R6=	0000000000000000	F6=	0000000.00000000
R7=	0000000000000000	F7=	0000000.00000000
R8=	0000000000000000	F8=	0000000.00000000
R9=	0000000000000000	F9=	0000000.00000000
R10=	0000000000000000	F10=	0000000.00000000
R11=	0000000000000000	F11=	0000000.00000000
R12=	0000000000000000	F12=	0000000.00000000
R13=	0000000000000000	F13=	0000000.00000000
R14=	0000000000000000	F14=	0000000.00000000
R15=	0000000000000000	F15=	0000000.00000000
R16=	0000000000000000	F16=	0000000.00000000
R17=	0000000000000000	F17=	0000000.00000000
R18=	0000000000000000	F18=	0000000.00000000
R19=	0000000000000000	F19=	0000000.00000000
R20=	0000000000000000	F20=	0000000.00000000
R21=	0000000000000000	F21=	0000000.00000000

Estrutura

MIPS (MIPS64) -Interface

Apresentação de informação estatística, útil para
otimização de código



The screenshot shows a window titled "Statistics" with a list of performance metrics. The metrics are grouped into three sections: "Execution", "Stalls", and "Code size". Each section lists a specific metric and its current value, which is 0 for all metrics except for "Code size" which is 40 Bytes.

```
Statistics
Execution
0 Cycles
0 Instructions

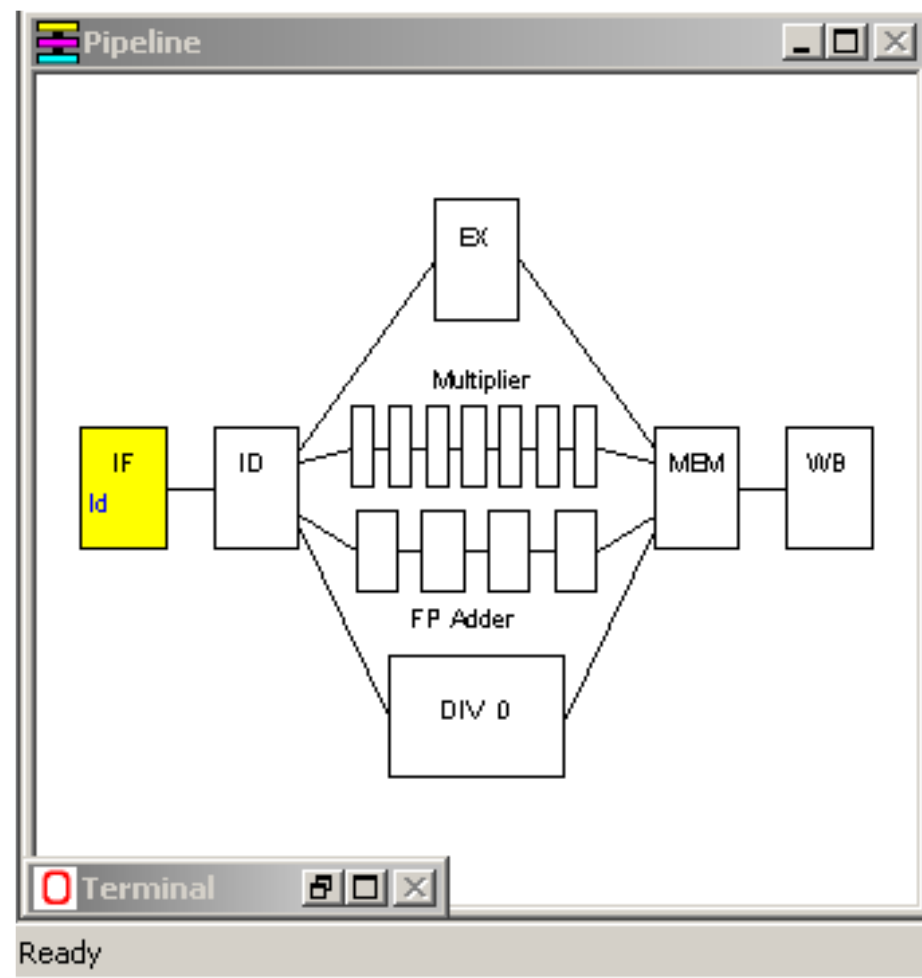
Stalls
0 RAW Stalls
0 WAW Stalls
0 WAR Stalls
0 Structural Stalls
0 Branch Taken Stalls
0 Branch Misprediction Stalls

Code size
40 Bytes
```

Estrutura

MIPS (MIPS64) -Interface

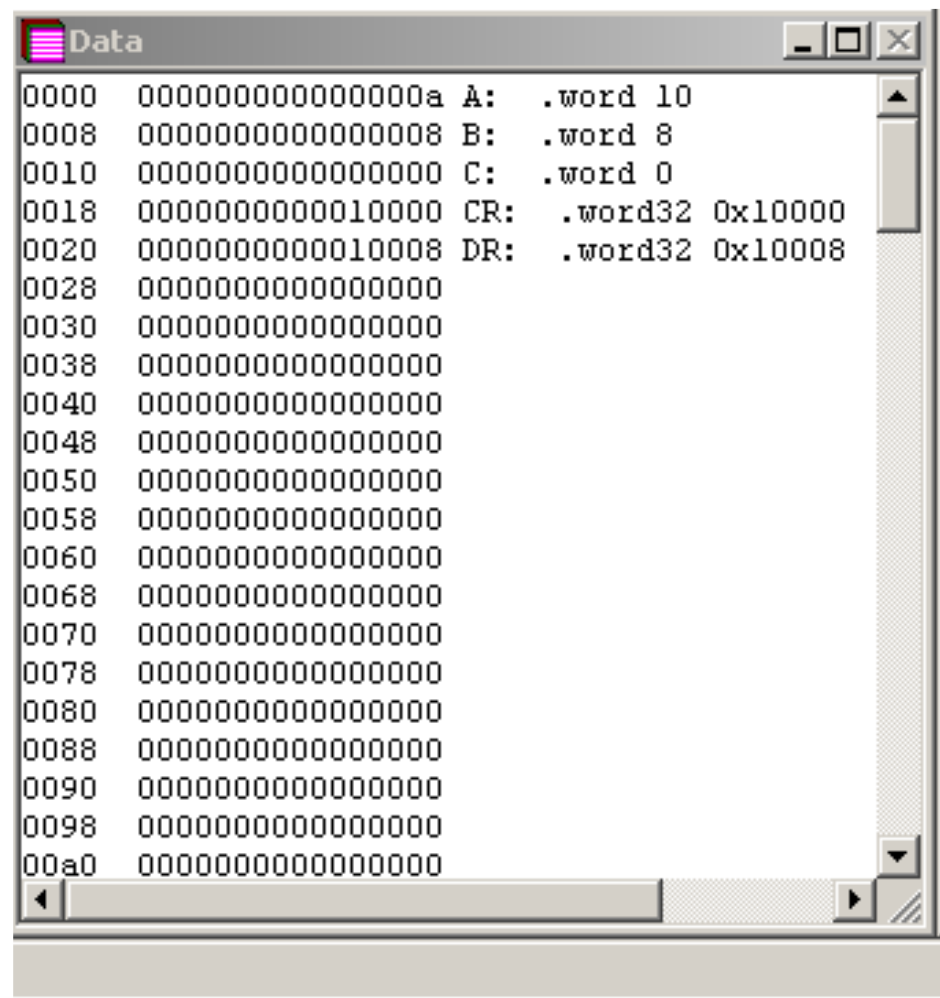
Esquema visual do pipeline, e da fase em que se encontra cada instrução



Estrutura

MIPS (MIPS64) -Interface

Estado da memória; valores iniciais e valores atuais das variáveis



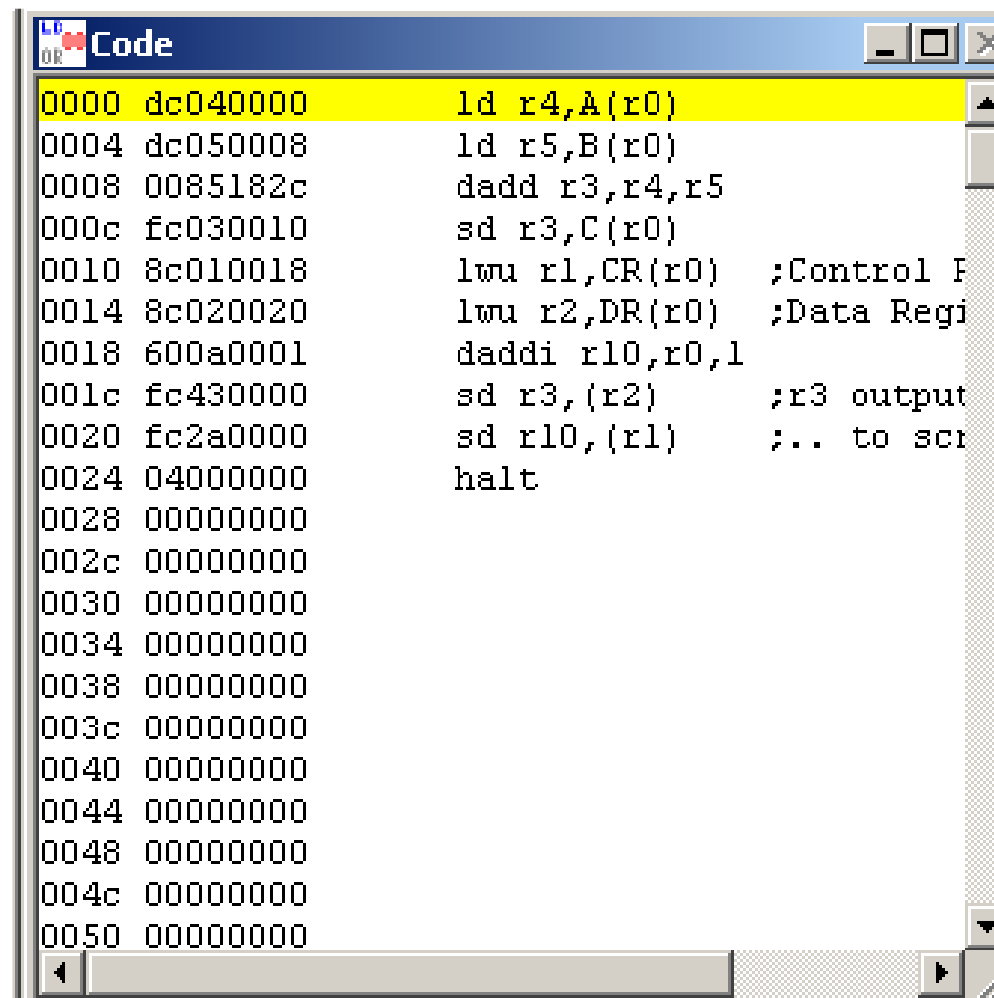
```
Data
0000 000000000000000a A: .word 10
0008 0000000000000008 B: .word 8
0010 0000000000000000 C: .word 0
0018 0000000000010000 CR: .word32 0x10000
0020 0000000000010008 DR: .word32 0x10008
0028 0000000000000000
0030 0000000000000000
0038 0000000000000000
0040 0000000000000000
0048 0000000000000000
0050 0000000000000000
0058 0000000000000000
0060 0000000000000000
0068 0000000000000000
0070 0000000000000000
0078 0000000000000000
0080 0000000000000000
0088 0000000000000000
0090 0000000000000000
0098 0000000000000000
00a0 0000000000000000
```

Estrutura

MIPS (MIPS64) -Interface

Área de Código

- 1)Endereço de cada instrução (4 hex.)
- 2)Codificação de cada instrução (8hex.)
- 3)Código Assembler

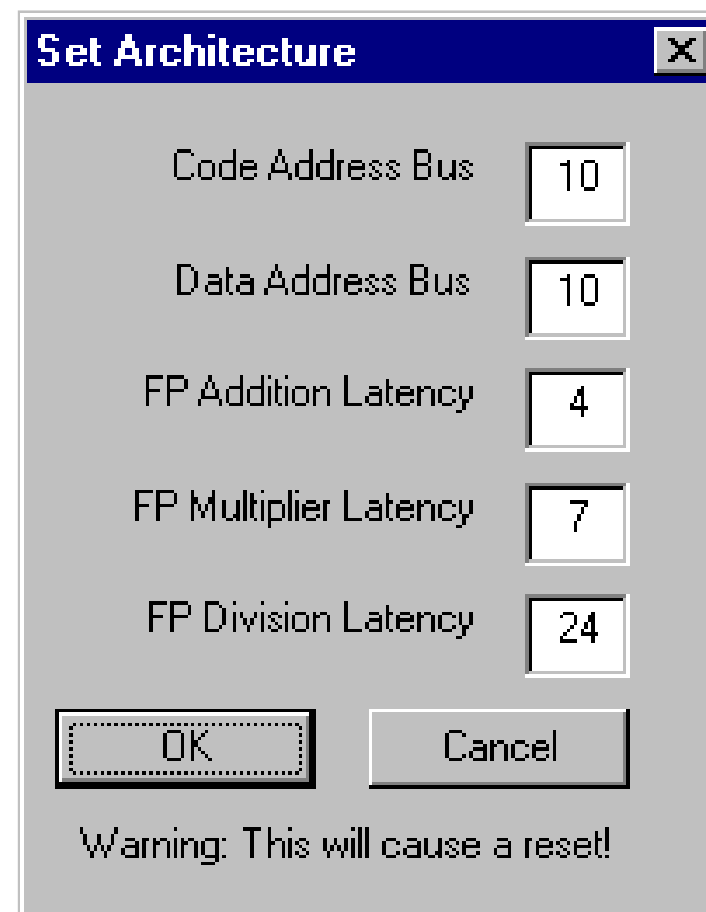


```
Code
0000 dc040000      ld r4,A(r0)
0004 dc050008      ld r5,B(r0)
0008 0085182c      dadd r3,r4,r5
000c fc030010      sd r3,C(r0)
0010 8c010018      lwu r1,CR(r0) ;Control F
0014 8c020020      lwu r2,DR(r0) ;Data Regi
0018 600a0001      daddi r10,r0,1
001c fc430000      sd r3,(r2) ;r3 output
0020 fc2a0000      sd r10,(r1) ;.. to scr
0024 04000000      halt
0028 00000000
002c 00000000
0030 00000000
0034 00000000
0038 00000000
003c 00000000
0040 00000000
0044 00000000
0048 00000000
004c 00000000
0050 00000000
```

Estrutura

MIPS (MIPS64) -Interface

Alterações na Arquitetura do processador
- Menu ***Configure-Architecture***



A screenshot of a 'Set Architecture' dialog box. The dialog has a title bar with a close button (X). It contains five input fields for configuration: 'Code Address Bus' (10), 'Data Address Bus' (10), 'FP Addition Latency' (4), 'FP Multiplier Latency' (7), and 'FP Division Latency' (24). At the bottom, there are 'OK' and 'Cancel' buttons. Below the buttons, a warning message reads: 'Warning: This will cause a reset!'.

Parameter	Value
Code Address Bus	10
Data Address Bus	10
FP Addition Latency	4
FP Multiplier Latency	7
FP Division Latency	24

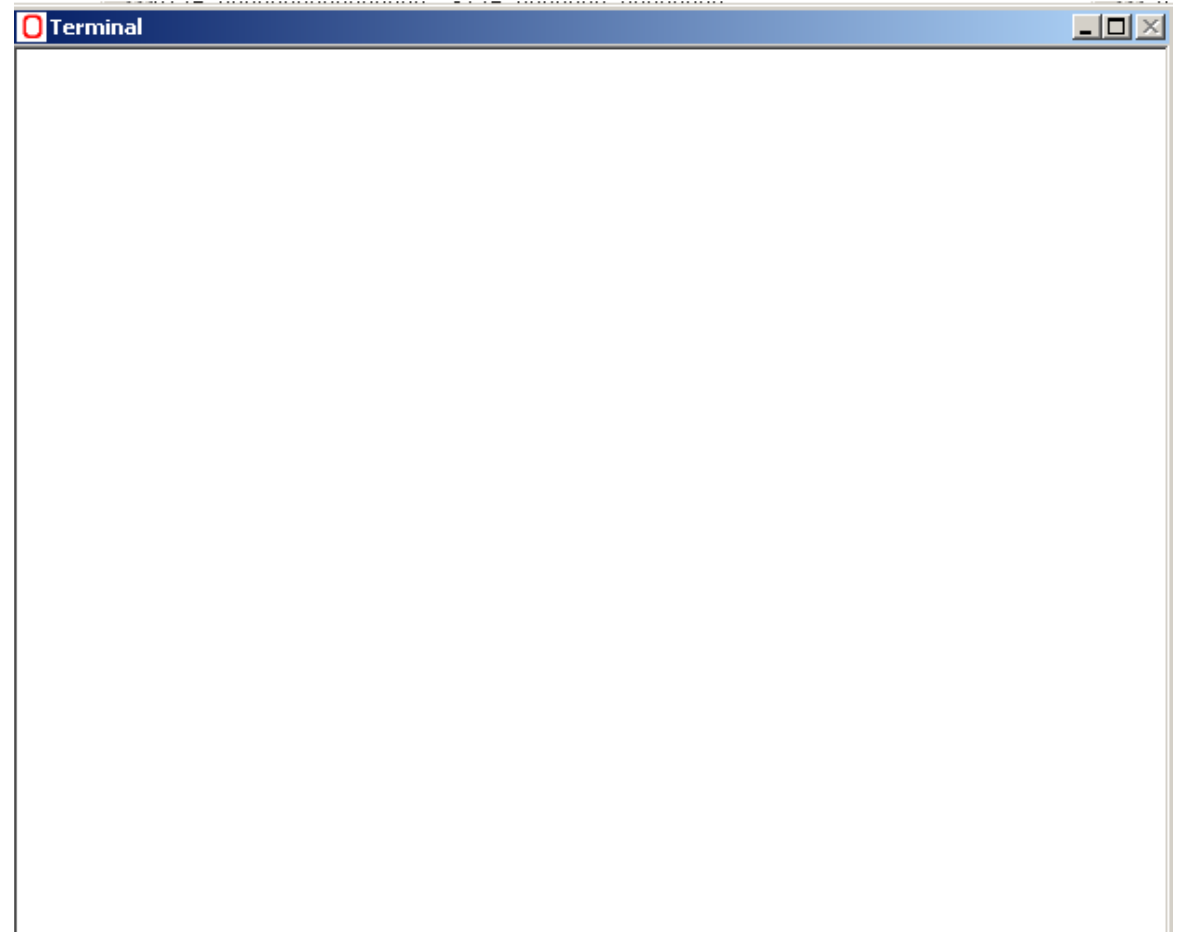
Warning: This will cause a reset!

Estrutura

MIPS (MIPS64) -Interface

Menu **Window-Terminal**

Terminal de interacção com o utilizador(**entrada e saída de dados**)



Estrutura

MIPS (MIPS64) -Interface

Na opção execute existem várias opções de execução do código
F7 –execução ciclo a ciclo

Reset

Volta ao início do código, mas não apaga a memória

Reload

Volta ao início do código e apaga a memória

Estrutura

MIPS (MIPS64) -Interface

<code>.data</code>	- início do segmento de dados
<code>.text</code>	- início do segmento de programa
<code>.code</code>	- o mesmo que <code>.text</code>
<code>.org <n></code>	- endereço inicial
<code>.space <n></code>	- reserva um espaço de n bytes na memória
<code>.ascii <s></code>	- define uma cadeia de caracteres terminada por nulo.
<code>.asciiz <s></code>	- define uma cadeia de caracteres
<code>.align <n></code>	- alinha o próximo endereço para uma fronteira de n-bytes
<code>.word <n1>,<n2>..</code>	- define palavras de 64 bits de dados com valor inicial
<code>.byte <n1>,<n2>..</code>	- define uma sequência de bytes com valor inicial.
<code>.word32 <n1>,<n2>..</code>	- define palavras de 32 bits com valor inicial.
<code>.word16 <n1>,<n2>..</code>	- define palavras de 16 bits com valor inicial.
<code>.double <f1>,<f2>..</code>	- define palavras no formato ponto-flutuante de 64 bits.

Onde <n> denota um número inteiro, <s> significa uma cadeia de caracteres entre aspas e <n1>, <n2> são valores inteiros separados por vírgula e <f1>, <f2> são valores reais separados por vírgula.

Estrutura

MIPS (MIPS64) -Interface

```
        .data
A:      .word 10
B:      .word 8
C:      .word 0
CR:     .word32 0x10000
DR:     .word32 0x10008

        .text
main:
    ld r4,A(r0)
    ld r5,B(r0)
    dadd r3,r4,r5
    sd r3,C(r0)

    lwu r1,CR(r0)    ;Control Register
    lwu r2,DR(r0)    ;Data Register
    daddi r10,r0,1
    sd r3,(r2)        ;r3 output..
    sd r10,(r1)        ;.. to screen

    halt
```

Estrutura

MIPS (MIPS64) -Interface

- Case sensitive:
 - diretivas (ex: .word, .data)
 - registos (ex: \$ra, \$zero)
 - variáveis
 - etiquetas
- NOT case sensitive:
 - registos (ex: r1, R1)
 - instruções (ex: dadd; DADD)
 - hexadecimal (ex: 0xba7; 0xBA7)

Case Sensitive		Not Case Sensitive	
Item	Exemplo	Item	Exemplo
Convenção de registos	\$ra, \$zero	Registos	R1, r1
Diretivas	.data, .word	instruções	daddi, DADDI
Etiquetas	NaoFaz:, Soma:		
Variáveis	DR:, a:, Nome:		

Estrutura

Tabela de Instruções e Diretivas

Instrução	Exemplo	Significado
Add	dadd R1, R2, R3	$R1 = R2 + R3$
Add Immediate	daddi R1, R2, #100	$R1 = R2 + 100$
Subtract	dsub R1, R2, R3	$R1 = R2 - R3$
And Logical Immediate	andi R1, R2, #20	$R1 = R2 \text{ AND } 20$
Or Logical	or R1, R2, R3	$R1 = R2 \text{ OR } R3$
Load (ld, lw, lb) d=64 bits w=32 bits b=8 bits	ld R1, A(R0)	$R1 = A$
	ld R1, A(R2)	$R1 = \text{Memória} [\text{Endereço de } A + R2]$
	ld R1, (R2)	$R1 = \text{Memória} [0 + R2]$
	ld R1, 200(R2)	$R1 = \text{Memória} [200 + R2]$
Store (ld, lw, lb)	sd R1, 200(R2)	$\text{Memória} [200 + R2] = R1$
Doubleword Shift Left Logical	dsl R1, R2, #3	$R1 = R2 \ll 2$ ($R1 = R2 * 2^3$)
Doubleword Shift Right Logical	dsr R1, R2, #4	$R1 = R2 \gg 4$ ($R1 = R2 / 2^4$)
Doubleword Shift Left Logical Variable	dslv R1, R2, R3	$R1 = R2 \ll R3$ ($R1 = R2 * 2^{R3}$)

Estrutura

Tabela de Instruções e Diretivas

Set Less Than	slt R1, R2, R3	Se $R2 < R3$ faça $R1 = 1$ senão $R1 = 0$
Set Less Than Immediate	slti R1, R2, #20	Se $R2 < 20$ faça $R1 = 1$ senão $R1 = 0$
Load Floating-point (64 bits)	l.d F1, 100(R1)	$F1 = \text{memória}[R1 + 100]$
Store Floating-point (64 bits)	s.d F2, 100(R1)	$\text{Memória}[R1 + 100] = F2$ (64 bits)
Add Double Precision	add.d F1, F2, F3	$F1 = F2 + F3$ (precisão dupla, 64 bits)
Subtract Single Precision	sub.s F1, F2, F3	$F1 = F2 - F3$ (precisão simples, 32 bits)
Multiply Double Precision	mul.d F1, F2, F3	$F1 = F2 * F3$ (precisão dupla)
Divide Single Precision	div.s F1, F2, F3	$F1 = F2 / F3$ (precisão simples)
Move Floating-point	mov.d F1, F2	$F1 = F2$ (precisão dupla)
Halt	halt	Para a execução do programa
No Operation	nop	Não faz nada

Estrutura

Tabela de Instruções e Diretivas

Saltos condicionais		Salta para etiqueta se ...
Branch Equal Zero	beqz R1, etiqueta	... se $R1 = 0$ (mesmo que beq R1,R0, etiqueta)
Branch on Not Equal Zero	bnez R1, etiqueta	... se $R1 \neq 0$ (mesmo que bne R1,R0, etiqueta)
Branch Equal	beq R1, R2, etiqueta	... se $R1 = R2$
Branch on Not Equal	bne R1, R2, etiqueta	... se $R1 \neq R2$
Saltos incondicionais		Salta para ...
Jump	j etiqueta	... etiqueta
Jump Register	jr R31	... a instrução no endereço guardado em R31
Jump and Link	jal etiqueta	... etiqueta e $R31 = PC + 4$ (chamada de procedimento)

Estrutura

Tabela de Instruções e Diretivas

Outras instruções:

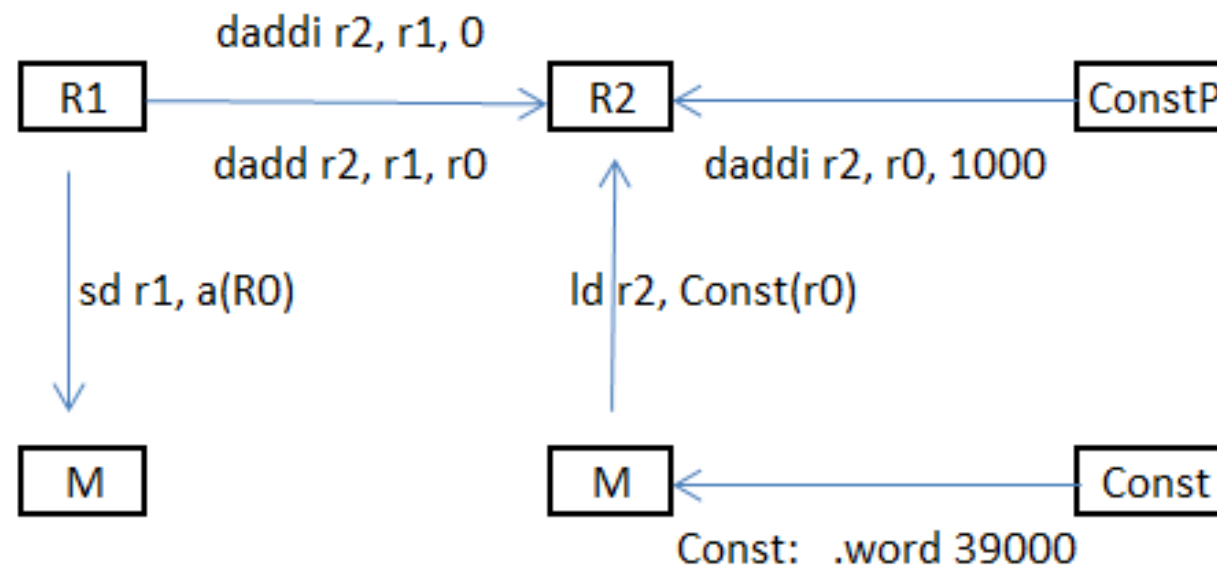
daddu - add integers unsigned	daddui - add immediate unsigned
lbu - load byte unsigned	dsubu - subtract integers unsigned
lh - load 16-bit half-word	sltu - set if less than unsigned
lhu - load 16-bit half word unsigned	sltiu - set if less than or equal imm. unsigned
sh - store 16-bit half-word	dsra - shift right arithmetic
lwu - load 32-bit word unsigned	dsrlv - shift right logical by variable
lui - load upper half of register immediate	dsrav - shift right arithmetic by variable
movz - move if register equals zero	and - logical and
movn - move if register not equal to zero	xor - logical xor
cvt.w.d - convert 32-bit integer to floating-point	ori - logical or immediate
cvt.l.d - convert 64-bit integer to floating-point	xori - exclusive or immediate
mtc1 - move 32 bit integer from integer register to floating-point register	xori - logical xor immediate

Estrutura

Movimentação de Dados

Inteiros

R	Registo
M	Memória
Const	Constante
ConstP	Constante < 2 ¹⁵
a	Variable im Mem



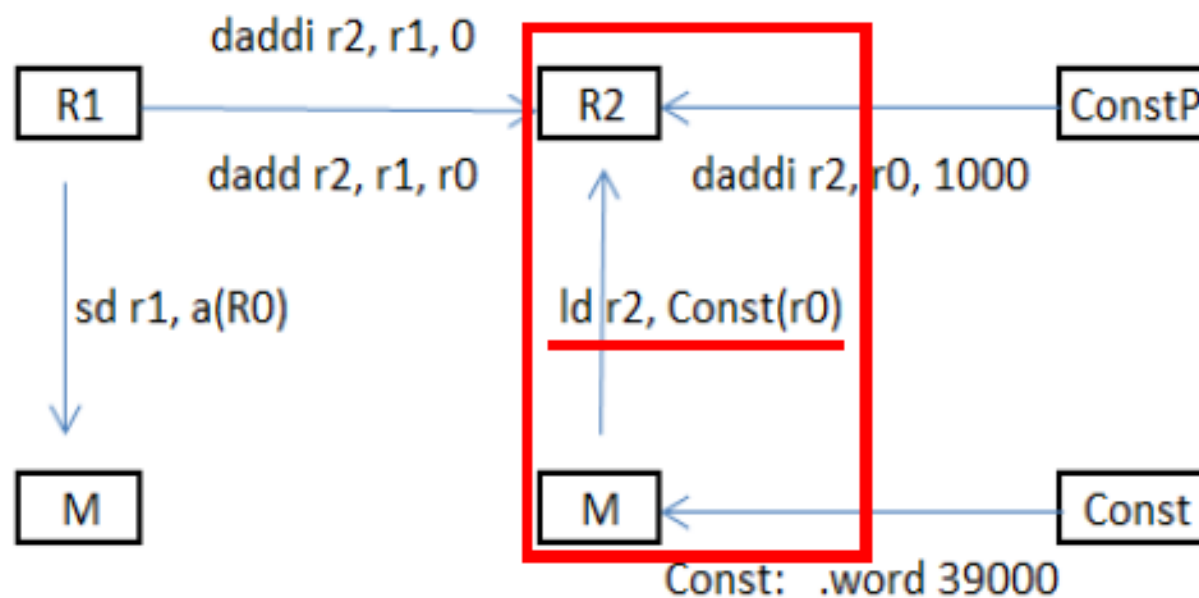
Estrutura

Movimentação de Dados

Carregar valor da **Memória**(variável) para **Registo**

Inteiros

R	Registo
M	Memória
Const	Constante
ConstP	Constante < 2 ¹⁵
a	Variable im Mem



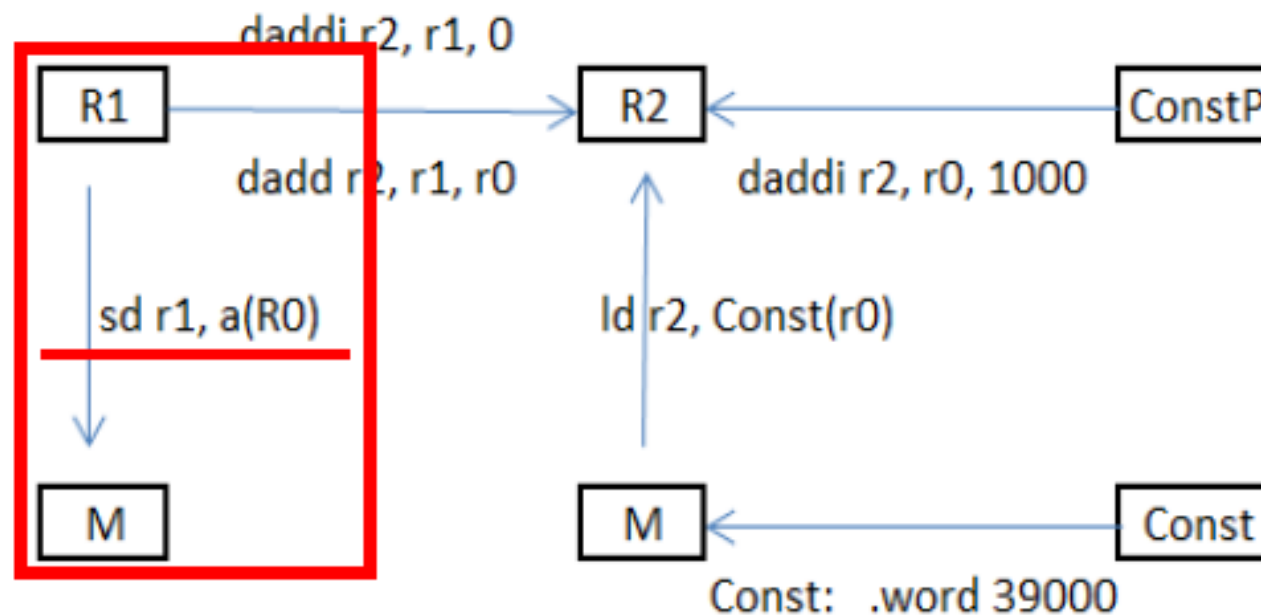
Estrutura

Movimentação de Dados

Carregar valor do **Registo** para a **Memória**

Inteiros

R	Registo
M	Memória
Const	Constante
ConstP	Constante < 2 ¹⁵
a	Variable im Mem



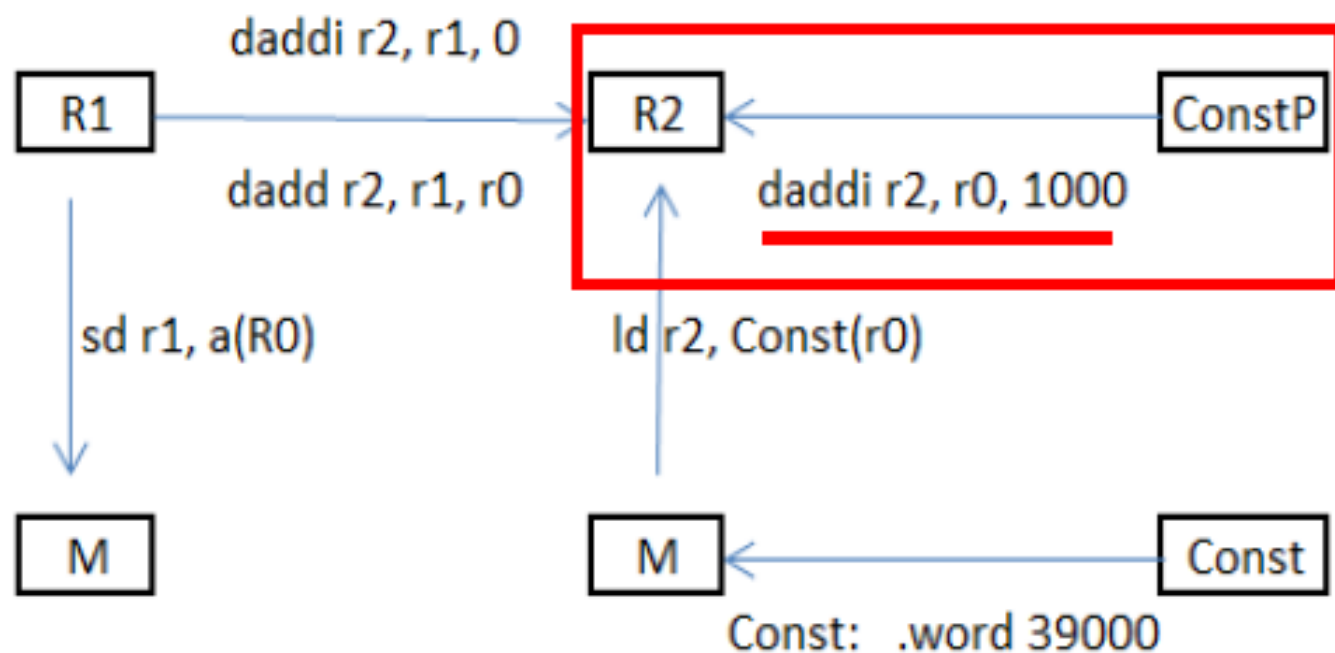
Estrutura

Movimentação de Dados

Colocar **Constante** com “valor pequeno” num **Registo**

Inteiros

R	Registo
M	Memória
Const	Constante
ConstP	Constante < 2 ¹⁵
a	Variable im Mem



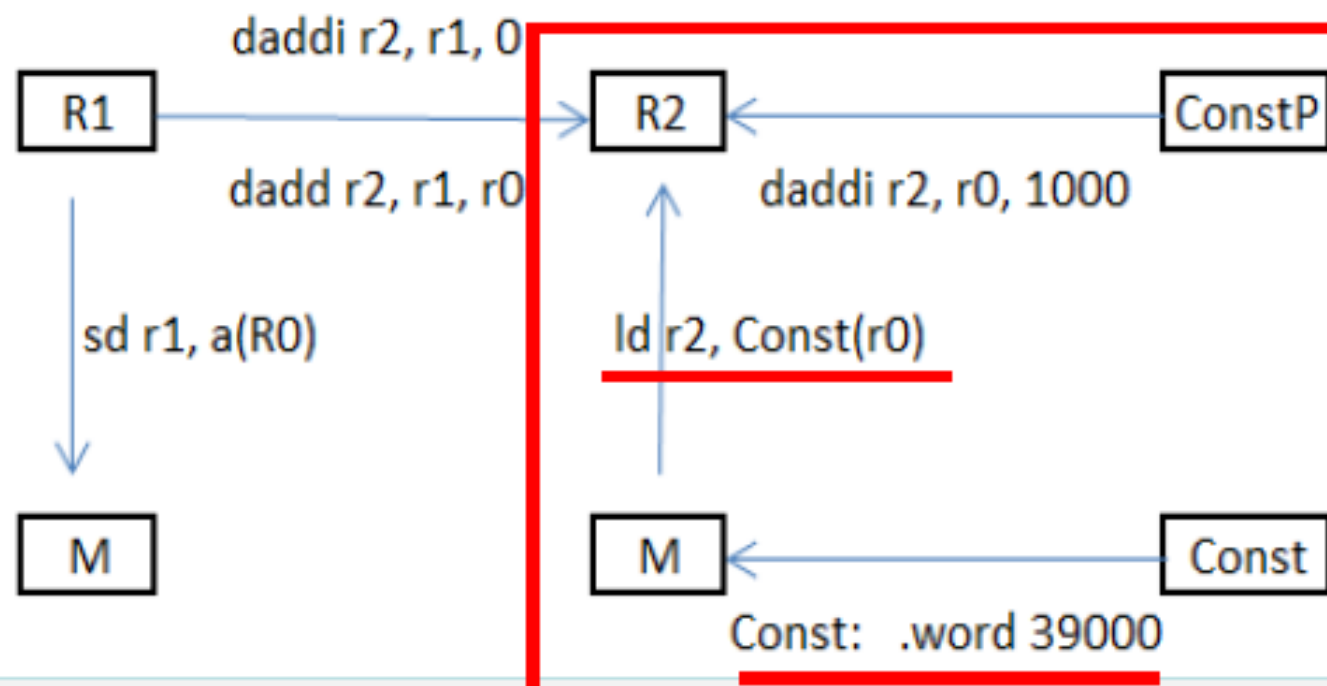
Estrutura

Movimentação de Dados

Colocar qualquer **Constante** num **Registo**

Inteiros

R	Registo
M	Memória
Const	Constante
ConstP	Constante < 2 ¹⁵
a	Variable im Mem



Estrutura

Movimentação de Dados

MemoryMappedI/O Area

Memory Mapped I/O area

Addresses of CONTROL and DATA registers

CONTROL: .word32 0x10000

DATA: .word32 0x10008

Set CONTROL = 1, Set DATA to Unsigned Integer to be output

Set CONTROL = 2, Set DATA to Signed Integer to be output

Set CONTROL = 3, Set DATA to Floating Point to be output

Set CONTROL = 4, Set DATA to address of string to be output

Set CONTROL = 5, Set DATA+5 to x coordinate, DATA+4 to y coordinate, and DATA to RGB colour to be output

Set CONTROL = 6, Clears the terminal screen

Set CONTROL = 7, Clears the graphics screen

Set CONTROL = 8, read the DATA (either an integer or a floating-point) from the keyboard

Set CONTROL = 9, read one byte from DATA, no character echo.

Estrutura

Movimentação de Dados

MemoryMappedI/O Area → Mostrar valor inteiro na janela “terminal” (enviar para écran)

```
.data
```

```
CR: .word32 0x10000 ; 0x é para indicar que é numero hexadecimal
```

```
DR: .word32 0x10008 ; 0x é para indicar que é numero hexadecimal
```

```
;r1:CR r2:DR r3:aux r4:aux
```

```
.code
```

```
lwu r1, CR(r0) ; Control Register
```

```
lwu r2, DR(r0) ; Data Register
```

Set CONTROL = 2, Set DATA to Signed Integer to be output

```
daddi r4,r0,37 ; enviar o valor 37 para DATA
```

```
sd r4,(r2)
```

```
daddi r3,r0,2 ; enviar código 2 para CODE (código para inteiro com sinal)
```

```
sd r3,(r1) ; chama rotina
```

```
halt
```

Estrutura

Movimentação de Dados

MemoryMappedI/O Area → Obter valor inteiro (ou de virgula flutuante) do teclado

```
.data
```

```
CR: .word32 0x10000 ; 0x é para indicar que é numero hexadecimal
```

```
DR: .word32 0x10008 ; 0x é para indicar que é numero hexadecimal
```

```
;r1:CR r2:DR r3:aux r4:aux
```

```
.code
```

```
lwu r1,CR(r0) ;Control Register
```

```
lwu r2,DR(r0) ;Data Register
```

Set CONTROL = 8, read the DATA (either an integer or a floating-point) from the keyboard

```
daddi r3,r0,8 ; Enviar 8 para CODE (código para ler dados de teclado)
```

```
sd r3,(r1) ; Chama rotina para a introdução de dados (usando teclado) em DATA
```

```
ld r4,(r2) ; Valor introduzido pelo teclado (ex: 347) que fica guardado em
```

```
 ; DATA é colocado em r4 por ex.
```

```
halt
```

Estrutura

Movimentação de Dados

- Obter inteiro (ou virgula flutuante) de teclado (CODE 8) :

- Mostrar inteiro s/ sinal (CODE 1) no écran:

```
.data
CR: .word32 0x10000 ; 0x é para indicar que é numero hexadecimal
DR: .word32 0x10008 ; 0x é para indicar que é numero hexadecimal
;r1:CR r2:DR r3:aux r4:aux
```

```
.code
lwu r1, CR(r0) ; Control Register
lwu r2, DR(r0) ; Data Register

daddi r3,r0,8 ; enviar código 8 para CODE
; (código para obter dados de teclado)

sd r3,(r1) ; chama rotina para obter valor do teclado
ld r4,(r2) ; valor obtido que fica em DATA é
; colocado em r4

halt
```

```
.code
lwu r1, CR(r0) ; Control Register
lwu r2, DR(r0) ; Data Register

daddi r4,r0,37 ; por ex. 37 que é para ser
; mostrado é colocado em r4
daddi r3,r0,1 ; enviar código 2 para CODE
; (código p/ inteiro s/sinal)

sd r4,(r2) ; valor 37 para DATA
sd r3,(r1) ; chama rotina para mostrar valor

halt
```

Estrutura

Movimentação de Dados

MemoryMappedI/O Area → Mostrar *string* no écran

.data

CR: .word32 0x10000
DR: .word32 0x10008
Mensagem: .asciiz "Arquitetura"

.text

lwu r1,CR(r0) ;Control Register
lwu r2,DR(r0) ;Data Register

daddi r3,r0,Mensagem
sd r3,(r2)
daddi r3,r0,4
sd r3,(r1)

Set CONTROL = 4, Set DATA to address of string to be output

Estrutura

Movimentação de Dados

MemoryMappedI/O Area → Limpar écran

```
.data
CR:      .word32 0x10000

.text
lwu r1,CR(r0) ;Control Register

daddi r3,r0,6
sd r3,(r1)
```

Set CONTROL = 6, Clears the terminal screen

Estrutura

Estrutura de Controlo

Caso	Estrutura	MIPS
1	Se $r1 = r2$ Faz op1;	bne r1, r2, NaoFaz op1 NaoFaz:
1a	Se $r1 \neq r2$ Faz op1;	beq r1, r2, NaoFaz op1 NaoFaz:
2	Se $r1 \geq r2$ Faz op1;	slt r3, r1, r2 bnez r3, NaoFaz op1 NaoFaz:
3	Se $r1 \leq r2$ Faz op1;	slt r3, r2, r1 bnez r3, NaoFaz op1 NaoFaz:

Estrutura

Estrutura de Controlo

4	Se $r1 < r2$ Faz op1;	<pre> slt r3, r1, r2 beqz r3, NaoFaz op1 NaoFaz: </pre>
4a	Se $r1 > r2$ Faz op1;	<pre> slt r3, r2, r1 beqz r3, NaoFaz op1 NaoFaz: </pre>
5	Se $r1 = r2$ Faz op1; Se não Faz op2; Duas soluções alternativas	<pre> beq r1, r2, Se op2; j FimSe Se: op1 FimSe: </pre>
		<pre> bne r1, r2, Else op1; j FimSe Else: op2 FimSe: </pre>

Estrutura

Estrutura de Controlo

6	<pre> se r1 = r2 ou r3 = r4 Faz op1; Se não Faz op2; </pre>	<pre> beq r1, r2, Se beq r3, r4, Se op2; j FimSe Se: Op1 FimSe: </pre>
6a	<pre> Se r1 = r2 E r3 = r4 Faz op1; Se não Faz op2; </pre>	<pre> bne r1, r2, Else bne r3, r4, Else op1; j FimSe Else: Op2 FimSe: </pre>
7	<pre> Para r5 desde 0 até r6 Faz op1; </pre>	<pre> daddi r5, r0, 0 For: slt r1, r5, r6 beqz r1, FimFor op1 daddi r5, r5, 1 j For FimFor: </pre>

Estrutura

Estrutura de Controlo

8	<pre> Enquanto r1 != r2 Faz op1; </pre>	<pre> While: beq r1, r2, FimWhile op1 j While FimWhile: </pre>
8a	<pre> Faz op1 Enquanto r1 != r2; </pre>	<pre> Faz: op1 bneq r1, r2, Faz </pre>
9	<pre> Se r1 = -3: Faz op1; stop; = 0: Faz op2; stop; = 43: Faz op3; stop; default: Faz op4; stop; daddi R2,R0,-3 beq R1,R2,FazOp1 beq R1,R0, FazOp2 daddi R2,R0,4 beq R1,R2, FazOp3 op4; j FimSwitch FazOp1: op1; j FimSwitch FazOp2: op2; j FimSwitch FazOp3: Op3; FimSwitch: </pre>	<pre> daddi r2, r0, -3 bne r1, r2, Testa0 op1; j FimSwitch Testa0: bnez r1, Testa43 op2; j FazOp3 Testa43: daddi r2, r0, 43 bne r1, r2, Default FazOp3: op3; j FimSwitch Default: op4 FimSwitch: Duas soluções alternativas </pre>

Consolidação de Conceitos

Variáveis vs. vetores em MIPS

- Em termos genéricos podemos ter:
 - **Variáveis**—estrutura que permite armazenar um único valor:
- Por exemplo: “**A = 1**” -a **variável A** armazena o valor **1**
 - **Vetores**—estrutura que permite armazenar um conjunto de valores, indexados por um índice que indica a posição:
 - Por exemplo: “**B[0] = 20**” —a **primeira posição** do **vetor B** armazena o valor **20**
- No MIPS não há lugar à existência de “**A**”, pelo que a variável é tratada como um vetor mas de um só elemento. Para tal utiliza-se o formato “**Vetor(Índice)**”:
 - Sabendo que o registo **r0** tem sempre o valor “**0**”, então um vetor que seja referenciado como “**A(r0)**”, significa que estamos a indexar o **índice 0** do **vetor A**

Consolidação de Conceitos MemoryMappedI/O Area

- Mostrar dados (valores inteiros) no écran (janela “terminal”):

Mostrar inteiro com sinal (CODE 2) no écran:

```
.data
CR: .word32 0x10000 ; 0x indica que é número hexadecimal – Control Register tem o endereço 10000 hex
DR: .word32 0x10008 ; 0x indica que é número hexadecimal – Data Register tem o endereço 10008 hex

.code
lwu r1, CR(r0) ; end. mem. do Control Register em r1
lwu r2, DR(r0) ; end. mem. do Data Register em r2

daddi r4,r0,37 ; valor 37 (que é p/ ser mostrado na
                ; janela “terminal”) é colocado em r4

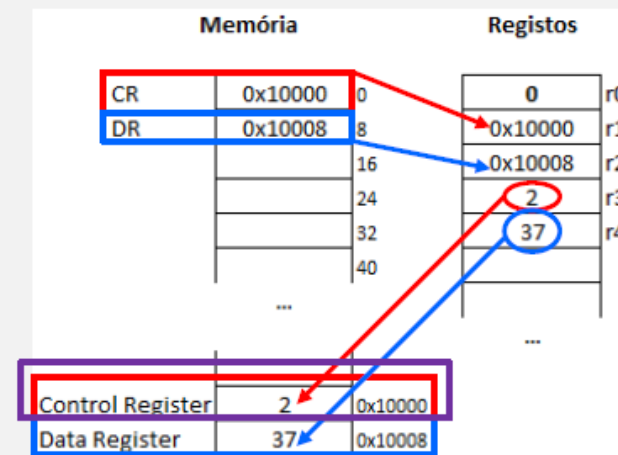
daddi r3,r0, 2 ; código 2 (código para mostrar inteiro
                ; com sinal) é colocado em r3

sd r4,(r2) ; valor a mostrar é colocado no Data
            ; Register (envia o valor 37 de r4 para o
            ; endereço de memória 10008 hex)

sd r3,(r1) ; chama rotina p/ mostrar valor (envia
            ; o código 2 de r3 para o end. de mem.
            ; 10000 hex - Control Register)

halt
```

Quando é colocado um valor numérico na zona de memória denominada “**Control Register**”, “pára tudo”, e é então invocada a rotina correspondente (neste caso é para mostrar dados numéricos – números inteiros com sinal)



Consolidação de Conceitos MemoryMappedI/O Area

- Obtenção dados de teclado (digitados na janela “terminal”):

Obter inteiro (ou Vírgula Flutuante) de teclado (**CODE 8**):

.data

CR: .word32 0x10000 ; 0x indica que é número hexadecimal – **Control Register** tem o **endereço 10000 hex**

DR: .word32 0x10008 ; 0x indica que é número hexadecimal – **Data Register** tem o **endereço 10008 hex**

.code

lwu r1, CR(r0) ; end. mem. do **Control Register** em r1

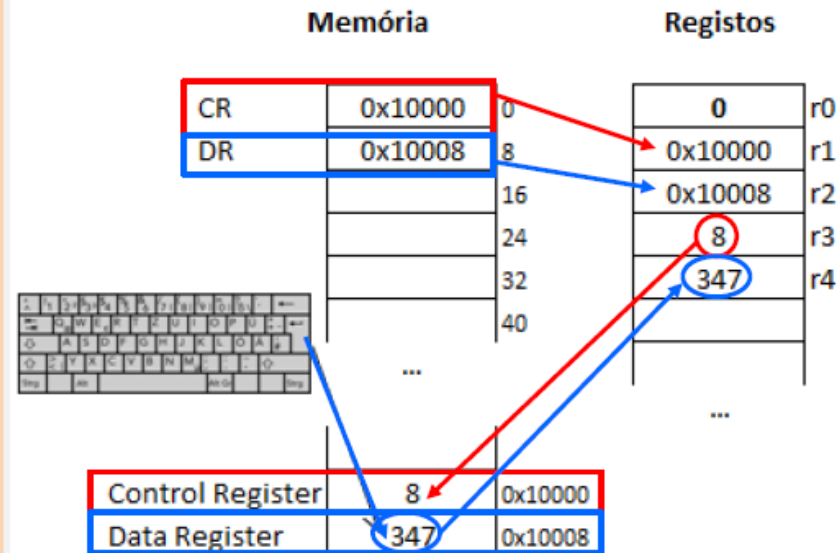
lwu r2, DR(r0) ; end. mem. do **Data Register** em r2

daddi r3,r0, 8 ; **código 8** (código para obter dados
; do teclado) é colocado em r3

sd r3,(r1) ; **chama rotina p/ obter valor** do teclado
; (envia o **código 8** de r3 para end. de
; memória 10000 hex - **Control Register**)

ld r4,(r2) ; **valor** digitado pelo teclado – e que fica
; guardado em **Memória (Data Register)**
; - é colocado em **Registo** (envia o **valor**
; **347** do end. de mem. 10008 hex p/ r4)

halt



Consolidação de Conceitos MemoryMappedI/O Area

- Obtenção dados de Teclado vs. Mostrar dados no écran:

Obter inteiro (ou V.F.) de teclado (**CODE 8**):

Mostrar inteiro s/ sinal (**CODE 1**) no écran:

.data

CR: .word32 0x10000 ; **Control Register** tem o endereço **10000**

DR: .word32 0x10008 ; **Data Register** tem o endereço **10008**

.code

lwu r1, CR(r0) ; end. mem. do **Control Register** em r1

lwu r2, DR(r0) ; end. mem. do **Data Register** em r2

daddi r3,r0, **8** ; **código 8** (código para obter dados
; do teclado) é colocado em r3

sd r3,(r1) ; chama rotina p/ obter valor do teclado
; (envia o **código 8** de r3 para end. de
; memória 10000 hex - **Control Register**)

ld r4,(r2) ; **valor** digitado pelo teclado – e que fica
; guardado em **Memória (Data Register)**
; - é colocado em **Registo** (envia o **valor**
; **347** do end. de mem. 10008 hex p/ r4

halt

.code

lwu r1, CR(r0) ; end. mem. do **Control Register** em r1

lwu r2, DR(r0) ; end. mem. do **Data Register** em r2

daddi r4,r0,**20** ; **valor 20** (que é p/ ser mostrado na
; janela “terminal”) é colocado em r4

daddi r3,r0, **1** ; **código 1** (código para mostrar inteiro
; sem sinal) é colocado em r3

sd r4,(r2) ; **valor a mostrar** é colocado no **Data**
; **Register** (envia o **valor 20** de r4 p/ o
; o endereço de memória 10008 hex)

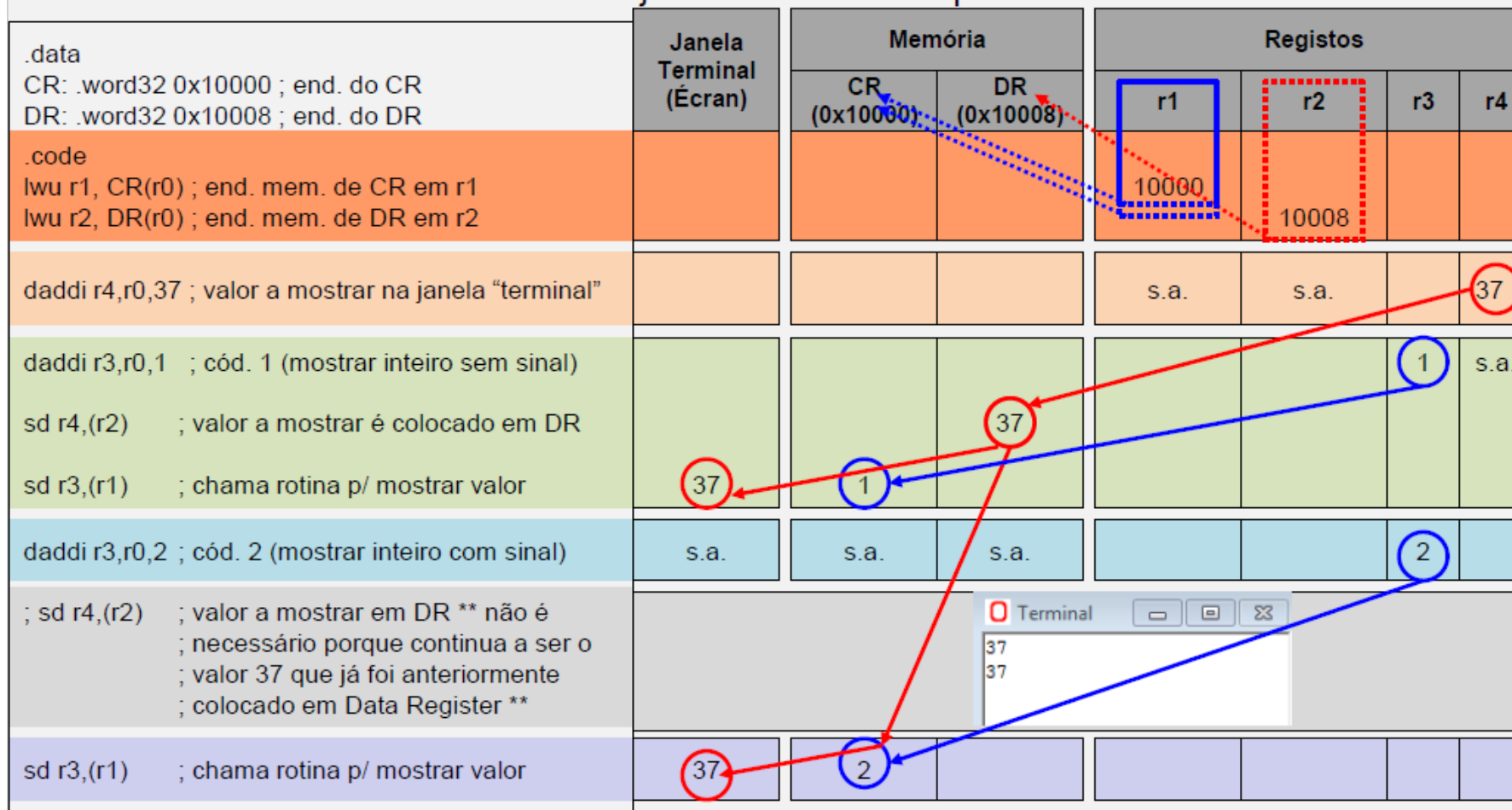
sd r3,(r1) ; chama rotina para mostrar
; **valor** (envia o **código 1** de r3
; para o endereço de memória
; 10000 hex - **Control Register**)

halt

Consolidação de Conceitos

MemoryMappedI/O Area

- Observando o código que se encontra à esquerda, tente compreender o comportamento de cada uma das estruturas MIPS e da janela "terminal" as quais estão identificadas à direita:

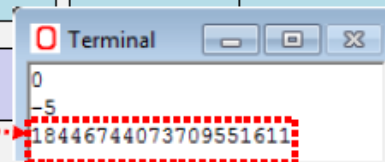


Consolidação de Conceitos MemoryMappedI/O Area

- Observando o código que se encontra à esquerda, tente descrever o comportamento de cada uma das estruturas MIPS e da janela “terminal” as quais estão identificadas à direita:

	Janela Terminal (Écran)	Memória		Registos			
		CR (0x10000)	DR (0x10008)	r1	r2	r3	r4
.data CR: .word32 0x10000 ; end. do CR DR: .word32 0x10008 ; end. do DR							
.code lwu r1, CR(r0) ; end. mem. de CR em r1 lwu r2, DR(r0) ; end. mem. de DR em r2				10000	10008		
daddi r3,r0,1 ; cód. 1 (mostrar inteiro sem sinal) sd r3,(r1) ; chama rotina p/ mostrar valor - não ; mostra nada, pois não há nada em DR	0			s.a.	s.a.	1	
daddi r4,r0,-5 ; valor a mostrar na janela “terminal”							-5
daddi r3,r0,2 ; cód. 2 (mostrar inteiro com sinal) sd r4,(r2) ; valor a mostrar é colocado em DR sd r3,(r1) ; chama rotina p/ mostrar valor	-5	2	-5			2	s.a.
daddi r3,r0,1 ; cód. 1 (mostrar inteiro sem sinal)	s.a.	s.a.	s.a.			1	
sd r3,(r1) ; chama rotina p/ mostrar valor	??	1					
halt							

Legenda: s.a. – sem alteração



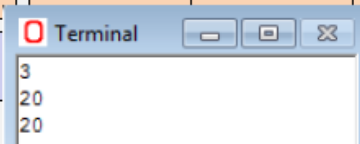
Consolidação de Conceitos

MemoryMappedI/O Area

- Observando o código que se encontra à esquerda, tente descrever o comportamento de cada uma das estruturas MIPS e da janela “terminal” as quais estão identificadas à direita:

	Janela Terminal (Écran)	Memória		Registos			
		CR (0x10000)	DR (0x10008)	r1	r2	r3	r4
.data CR: .word32 0x10000 ; end. do CR DR: .word32 0x10008 ; end. do DR							
.code lwu r1, CR(r0) ; end. mem. de CR em r1 lwu r2, DR(r0) ; end. mem. de DR em r2				10000	10008		
daddi r4,r0,3 ; valor a mostrar na janela “terminal”				s.a.	s.a.		3
daddi r3,r0,1 ; cód. 1 (mostrar inteiro sem sinal)						1	s.a.
sd r4,(r2) ; valor a mostrar é colocado em DR			3				
sd r3,(r1) ; chama rotina p/ mostrar valor	3	1					
daddi r3,r0,8 ; cód. 8 (obter valor)	s.a.	s.a.	s.a.			8	
sd r3,(r1) ; chama rotina p/ obter valor		8					
ld r4,(r2) ; valor digitado pelo teclado (20)	20		20				20
daddi r3,r0,1 ; cód. 1 (mostrar inteiro sem sinal)	s.a.	s.a.	s.a.			1	s.a.
sd r3,(r1) ; chama rotina p/ mostrar valor	20	1					
halt							

Legenda: s.a. – sem alteração



Exercícios

- Exercício 1

Elabore um programa em assembler (MIPS64) que implemente o seguinte código escrito em linguagem C:

a)

```
int c, a=10, b=8;
```

```
c = a + b;
```

b)

```
int c, a=10, b=8, z=2;
```

```
c = a + b + z;
```

Exercícios

- Exercício 2

Elabore um programa em assembler (MIPS64) que troque o valor de 2 variáveis definidas em memória:

- a) Utilizando apenas referências de memória
- b) Efetuando a cópia para os registos e efetuando a troca através dos mesmos

Exercícios

- Exercício 3

Adapte o exercício 1 de forma a enviar o resultado para o terminal.

Exercícios

- Exercício 4

Adapte o exercício 3 de forma a obter do utilizador os valores a somar.

Exercícios Extras

- Exercício 5

Elabore um programa em assembler (MIPS64) que:

a) Mostre a mensagem “HelloWorld!”

b) Obtenha 2 valores a e b a partir do teclado. Se a e b forem iguais escreva a mensagem “Iguais!”

c) Obtenha 2 valores a e b a partir do teclado:

- Se a e b forem iguais escreva a mensagem “Iguais!” (alínea anterior)
- Caso contrário, escreva a mensagem “O maior é: ” seguido do maior dos valores.

Exercícios Extras

- Exercício 6

Elabore um programa em assembler (MIPS64) que obtenha de um utilizador um valor secreto. Depois de se apagar o terminal um segundo utilizador vai tentar adivinhar o número. O programa vai informando se o n.º secreto é maior ou menor.

a) No final, depois de dar os parabéns, imprime o n.º de tentativas.

b) Imprima mensagens de parabéns diferentes consoante o n.º de tentativas:

≤2, “Fantástico!”

≤4, “Muito Bom!”

>4, “Já não era sem tempo...”

Exercícios Extras

- Exercício 7

Utilize a rotina fornecida “**ex07_Rotina_Bin.s**”, para mostrar o valor em binário em cada alínea.

a) Considere que o estado de uma porta é representado por “0” se “aberta”, e “1” se “fechada”, e que o estado das portas “0” a “7” é guardado num byte pela seguinte ordem: 76543210.

- 1) Considere que apenas as portas “0” e “5” estão abertas. Coloque o valor correspondente em r1.
- 2) Abra a porta “2”
- 3) Feche a porta “5”
- 4) Altere o estado das portas “1” e “2”

b) Um endereço IPv4 é um identificador de 32 bits e é representado no formato *dotteddecimal*.

- 1) Declare um vetor de bytes que permita guardar o IP **192.168.10.15** (o MIPS é *littleendian*)
- 2) Declare uma máscara de rede **/25** (25 bits a “1” e os restantes a “0”) que separa os identificadores da rede e do *host*
- 3) Obtenha o endereço da sub-rede (parte do *host* a “0”)
- 4) Obtenha o endereço de *Broadcast* (parte do *host* a “1”)

Exercícios Extras

- Exercício 8

Elabore um programa em assembler (MIPS64) que:

1)Declare um vetor com os seguintes 10 inteiros de 64 bits:

3, 15, 100, 7, 0xf9876543987625aa,17, 5, 15, -100, 2023

2)Mostre os elementos do vetor

3)Some “1” ao 4º elemento do vetor

* Reescreva o programa para trabalhar com os seguintes inteiros de 16 bits:

3, 15, 100, 7, 0xf5aa, 17, 5, 15, -100, 2023

Exercícios Extras

- Exercício 9

Elabore programas em assembler (MIPS64) que:

a) Conte:

- 1) Os caracteres de uma *string*
- 2) A ocorrência de espaços numa *string*

b) Faça as seguintes ações:

- 1) Obtenha do utilizador um carácter minúsculo e mostre no ecrã a maiúscula correspondente
- 2) Converta as minúsculas de uma *string* para as maiúsculas correspondentes

c) Converta um inteiro contido numa *string* para inteiro, de modo a poder ser utilizado em operações aritméticas

Exercícios Extras

- Exercício 10

Elabore uma rotina Max em assembler MIPS64 que receba 2 inteiros como parâmetros de entrada, e devolva o maior.

a)Elabore um programa que permita testar a rotina.

b)Passe os parâmetros de entrada pela pilha.

c)Utilize uma variável local (na pilha):

`i = 6;`

`i++;`

d)Suponha que a rotina necessita utilizar muitos registos pelo que tem que recorrer ao registo \$s0 dentro da rotina. Proceda de acordo com a convenção MIPS, guarde o valor do registo na pilha no início da rotina, utilize depois o registo e restabeleça o seu valor antes do final da rotina.

e)Passe também o resultado pela pilha

f)Escreva um programa que chama uma Rotina1, que pede os valores ao utilizador, chama a rotina Max, e mostra o maior (deve guardar o valor de \$ra na pilha no início da Rotina1e restabelecer o seu valor no final !).